

EduSaaS AI Proctoring System

Technical Documentation

AI-based Online Examination Monitoring and Fraud Detection

1. Project Overview

EduSaaS AI Proctoring is a real-time online examination monitoring system. The browser captures webcam frames and sends them to a Python FastAPI backend through WebSocket. The backend applies computer-vision detectors and a rule-based fraud engine, then sends the decision back to the browser.

The service is named 'EduSaaS AI Proctoring Service' and exposes the /ws/proctor WebSocket endpoint.

2. Objectives

- Monitor the student's webcam during an examination.
- Detect face presence and facial landmarks.
- Perform eye, blink, head-pose and mouth analysis as supporting information.
- Detect mobile-phone presence.
- Monitor tab switching and fullscreen exit.
- Use persistence so continuous detections are not counted frame-by-frame.
- Generate warning, pause and termination decisions.
- Return proctoring results to the browser in real time.

3. Technologies Used

Technology	Purpose	Usage
Python	Backend	AI-proctoring service and orchestration
FastAPI	Web framework	HTTP API and WebSocket endpoint
Uvicorn	ASGI server	Runs FastAPI
WebSocket	Real-time communication	Frames, browser events and results
OpenCV	Computer vision	Frame decoding/processing
NumPy	Numerical processing	Image arrays and numerical values
MediaPipe	Facial computer vision	Face/landmark-based analysis
YOLO11s	Object detection	Mobile-phone detection
JavaScript	Browser client	Camera, browser events and UI callbacks
HTML5 Camera APIs	Camera access	Webcam capture
Fullscreen API	Browser monitoring	Detect fullscreen changes
Page Visibility API	Browser monitoring	Detect tab/page switching
Base64	Data encoding	Transfers encoded camera frames

4. File-by-File Explanation

The supplied main.py imports and initializes the following detector modules. Their descriptions below are limited to the roles supported by the supplied source.

main.py: Central backend. Creates FastAPI, initializes detectors, receives WebSocket messages and video frames, runs analysis, applies fraud rules and sends results.

face_presence.py: Face-presence detector. Determines whether a face is visible and provides face-count/status information.

face_mesh.py: Facial-landmark detector. Supplies landmarks used by eye, blink, head-pose and mouth analysis.

eye_tracking.py: Determines eye direction from landmarks. Under the requested policy it is informational and is not a fraud violation.

blink_detection.py: Detects blinking from landmarks. It remains an AI output but is not counted as a fraud violation.

head_pose.py: Estimates head orientation. It is retained for monitoring but is not counted as a fraud violation.

mouth_detection.py: Performs mouth-related facial analysis using landmarks.

phone_detection.py: Detects mobile phones in camera frames; the supplied service identifies YOLO11s for this detector.

fraud_engine.py: Decision layer for persistence, counters, warnings, pauses and termination. It should consume detector results rather than perform vision detection.

main_updated.py: Generated updated backend file from the recent modification; merge its intended changes into the actual project main.py before deployment.

Proctoring JavaScript client: Browser-side controller that starts the camera, opens WebSocket, sends frames, monitors browser events and handles warning/pause/termination results.

5. System Architecture

1. Student browser starts camera and browser monitoring.
2. JavaScript establishes a WebSocket connection with FastAPI.
3. Camera frames are encoded and sent as VIDEO_FRAME messages.
4. FastAPI decodes each frame using Base64, NumPy and OpenCV.
5. AI detectors analyze the frame.
6. Relevant detection evidence is passed to the fraud-decision layer.
7. The fraud engine applies persistence and violation thresholds.
8. The backend returns a JSON-safe PROCTORING_RESULT.
9. The browser displays warnings, pauses the exam when required, or terminates it.

6. AI Detection Workflow

A frame is analyzed by face detection, face mesh, eye tracking, blink detection, head-pose detection, mouth detection and phone detection. The supplied source initializes all seven detector components before accepting examination traffic.

MediaPipe is identified for the face-related components and YOLO11s for phone detection.

7. Fraud Policy

Violation	1st	2nd	3rd	4th
Mobile phone	Warning + 10s pause	Warning + 10s pause	Terminate	-
Face missing	Warning	Warning	Terminate	-
Fullscreen exit	Warning	Warning	Warning	Terminate
Tab switch	Warning	Warning	Warning	Terminate
Head pose	Ignore	Ignore	Ignore	Ignore
Eye tracking	Ignore	Ignore	Ignore	Ignore
Blink	Ignore	Ignore	Ignore	Ignore

8. Mobile-Phone Handling

The first two persistent mobile-phone violations produce a warning and a 10-second exam pause. The frontend should display a countdown of 10, 9, 8, ... 1 and then resume the examination. The third persistent mobile-phone violation terminates the examination.

9. Face Handling

A continuous missing-face condition is treated as a face violation only after the configured persistence interval. The first two violations produce warnings; the third terminates the examination.

10. Fullscreen and Tab Switching

The browser uses browser APIs to detect visibility changes and fullscreen changes. These events are sent to the backend as browser-violation messages. A short event cooldown prevents closely related events from being counted twice. The requested final rule allows three warnings and terminates on the fourth browser violation.

11. Head Pose, Eye Tracking and Blink

These detectors may continue running and their outputs can be included in the proctoring result, but they do not contribute to fraud counters under the requested policy.

12. Violation Persistence

Camera frames arrive continuously, so one real-world event can generate many identical detections. Persistence prevents each frame from becoming a new violation. A relevant AI condition must remain continuously detected for the configured persistence interval before one violation is recorded.

This is particularly important for phone and missing-face detection.

13. WebSocket Message Flow

The main endpoint is /ws/proctor. The browser sends START_EXAM, STOP_EXAM, BROWSER_VIOLATION and VIDEO_FRAME messages. The backend returns PROCTORING_STARTED, PROCTORING_RESULT, PROCTORING_STOPPED, PROCTORING_ERROR and EXAM_TERMINATED messages.

10. START_EXAM -> backend starts the logical proctoring session.
11. VIDEO_FRAME -> backend decodes and analyzes the frame.
12. BROWSER_VIOLATION -> backend evaluates tab/fullscreen policy.
13. PROCTORING_RESULT -> backend returns AI and fraud information.
14. PAUSE_EXAM -> frontend pauses the exam and runs the 10-second countdown.
15. EXAM_TERMINATED -> frontend ends the examination.

14. JSON Result

The backend combines face, face mesh, eyes, blink, head, mouth, phone and fraud data into one JSON-safe PROCTORING_RESULT. A JSON-safe conversion function is required because MediaPipe and NumPy can produce objects that cannot be sent directly through WebSocket JSON serialization.

15. End-to-End Workflow

16. Student opens the exam.
17. Browser requests camera access.
18. Camera stream starts.
19. WebSocket connection is established.
20. Fullscreen and browser monitoring are enabled.
21. Video frames are sent continuously.
22. Python decodes each frame.
23. AI detectors generate evidence.
24. Fraud engine evaluates relevant evidence.
25. Persistence prevents repeated counting.
26. Browser events are evaluated separately.
27. Backend sends NORMAL, WARNING, PAUSE_EXAM or TERMINATE_EXAM.
28. Frontend displays the corresponding result.
29. For mobile warnings, the exam pauses for 10 seconds and resumes.
30. For the configured third/ fourth violations, the exam is terminated according to the violation type.

16. AI/ML and Rule-Based Components

The system contains both AI/computer-vision and deterministic rule-based components. MediaPipe and YOLO11s are used for detection. The fraud engine applies explicit rules to those detection results; it is not itself an ML model.

- AI/computer vision: face, landmarks and mobile detection.
- Rule-based layer: persistence, counters, warnings, pauses and termination.
- Real-time layer: WebSocket.
- Browser layer: JavaScript and browser monitoring APIs.

17. Backend Endpoints

Endpoint	Type	Purpose
/	HTTP GET	Returns service/status information.
/health	HTTP GET	Returns basic service health.
/ws/proctor	WebSocket	Main real-time examination proctoring channel.

18. Important Implementation Note

The supplied original main.py still contains the older generic fraud policy in some sections, where the first AI violation warns and the second terminates. Therefore, the policy table in this document represents the requested final design. The actual project main.py and fraud_engine.py must contain the updated thresholds before the documentation is treated as evidence of the deployed behavior.

19. Conclusion

EduSaaS AI Proctoring combines browser monitoring, real-time WebSocket communication, computer-vision detection and a rule-based fraud engine. The architecture separates detection from decision-making. This allows AI models to provide evidence while the fraud engine applies the examination policy.

The requested final policy reduces unnecessary termination: mobile and missing-face violations require repeated persistent events, fullscreen and tab switching allow three warnings before termination, and head pose, eye tracking and blink detection do not create fraud violations.